UCS420: Cognitive Computing

ASSIGNMENT 6 — NUMPY-II

Name: Krish Marwaha
Roll No.: 1024170223

Q1. Given temperature = np.array([25, 28, 31, 35, 38, 27, 33, 40]) and an average measurement error of +2°C: (a) add 2°C using vectorization, (b) convert corrected temperatures to Fahrenheit, (c) select readings above 32°C using Boolean indexing, (d) count them, and (e) explain the efficiency.

Answer / Python Code

```
import numpy as np
temperature = np.array([25, 28, 31, 35, 38, 27, 33, 40])
corrected = temperature + 2
fahrenheit = (9 / 5) * corrected + 32
above_32 = corrected[corrected > 32]
count_above_32 = np.sum(corrected > 32)
print("Corrected temperatures:", corrected)
print("Temperatures in Fahrenheit:", fahrenheit)
print("Readings above 32 C:", above_32)
print("Count above 32 C:", count_above_32)
```

Output

```
Corrected temperatures: [27 30 33 37 40 29 35 42]
Temperatures in Fahrenheit: [ 80.6  86.   91.4  98.6 104.   84.2  95.  107.6]
Readings above 32 C: [33 37 40 35 42]
Count above 32 C: 5
```

(e) Vectorization applies operations to an entire NumPy array without an explicit Python loop. Boolean indexing selects values satisfying a condition. Both make numerical code concise and typically more efficient.

Q2. For the given daily step matrix, calculate total, mean, maximum, minimum, total per day (axis=0), total per user (axis=1), and the position of the maximum using argmax().

Answer / Python Code

```
import numpy as np
steps = np.array([[5000, 6200, 7100],
                  [8000, 7500, 9000],
                  [4500, 5100, 4800],
                  [9000, 8500, 9500]])
print("Total steps:", np.sum(steps))
print("Mean steps:", np.mean(steps))
print("Maximum steps:", np.max(steps))
print("Minimum steps:", np.min(steps))
print("Total per day:", np.sum(steps, axis=0))
print("Total per user:", np.sum(steps, axis=1))
print("Maximum position (row, column):", np.unravel_index(np.argmax(steps), steps.shape))
```

Output

```
Total steps: 84200
Mean steps: 7016.67
Maximum steps: 9500
Minimum steps: 4500
Total per day: [26500 27300 30400]
Total per user: [18300 24500 14400 27000]
Maximum position (row, column): (np.int64(3), np.int64(2))
```

Q3. Demonstrate NumPy slicing views, copy(), reshape, indexing, flatten(), ravel(), and array properties for [1, 2, 3, 4, 5, 6].

Answer / Python Code

```
import numpy as np
original = np.array([1, 2, 3, 4, 5, 6])
subset = original[1:5]
subset[0] = 999
print("After changing subset:", original, subset)

original2 = np.array([1, 2, 3, 4, 5, 6])
copied = original2[1:5].copy()
copied[0] = 500
print("After changing copy:", original2, copied)

matrix = np.arange(1, 13).reshape(3, 4)
print("Matrix:\n", matrix)
print("First row:", matrix[0, :])
print("Last row:", matrix[-1, :])
print("Second column:", matrix[:, 1])
print("Rows 1-2, columns 2-3:", matrix[1:3, 2:4])

raveled = matrix.ravel()
raveled[0] = 100
print("After ravel change:\n", matrix)
flattened = matrix.flatten()
flattened[1] = 200
print("After flatten change:\n", matrix)
print("flattened:", flattened)
print("shape:", matrix.shape)
print("ndim:", matrix.ndim)
print("size:", matrix.size)
print("dtype:", matrix.dtype)
```

Output

```
After changing subset: [  1 999   3   4   5   6] [999   3   4   5]
After changing copy: [1 2 3 4 5 6] [500   3   4   5]
Matrix:
 [[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
First row: [1 2 3 4]
Last row: [ 9 10 11 12]
Second column: [ 2  6 10]
Rows 1-2, columns 2-3: [[ 7  8]
 [11 12]]
After ravel change:
 [[100   2   3   4]
 [ 5   6   7   8]
 [ 9 10 11 12]]
After flatten change:
 [[100   2   3   4]
 [ 5   6   7   8]
 [ 9 10 11 12]]
flattened: [100 200   3   4   5   6   7   8   9 10 11 12]
shape: (3, 4)
ndim: 2
size: 12
dtype: int64
```

Explanation: Basic slicing creates a view here, so changing subset also changes original. copy() is independent. ravel() returns a view when possible, while flatten() returns a copy.

Q4. Use the supplied X and y arrays for a cognitive-assistive system. Show shape and dimensions, X.T, X.T @ X, inverse where appropriate, OLS coefficients, explain coefficients, and predict for new_user = [5, 40, 7].

Answer / Python Code

```
import numpy as np
X = np.array([[6,70,3], [5,50,6], [8,80,2], [4,30,8]], dtype=float)
y = np.array([40,65,30,85], dtype=float)
print("Shape of X:", X.shape)
print("Dimensions of X:", X.ndim)
print("X transpose:\n", X.T)
xtx = X.T @ X
print("X.T @ X:\n", xtx)
try:
    beta = np.linalg.inv(xtx) @ X.T @ y
except np.linalg.LinAlgError:
    beta = np.linalg.pinv(X) @ y
print("Coefficients:", beta)
new_user = np.array([5,40,7], dtype=float)
print("Predicted assistance score:", new_user @ beta)
```

Output

```
Shape of X: (4, 3)
Dimensions of X: 2
X transpose:
 [[ 6.  5.  8.  4.]
 [70. 50. 80. 30.]
 [ 3.  6.  2.  8.]]
X.T @ X:
 [[  141.  1430.    96.]
 [ 1430. 14700.   910.]
 [   96.   910.   113.]]
Coefficients (least-squares): [-0.15921454  0.14083853 10.06302242]
Predicted assistance score: 75.2786
```

(f) Each coefficient estimates the change in predicted assistance score for a one-unit increase in the corresponding feature, holding other features constant. Features are sleep hours, activity level, and stress level. The specified equation has no intercept.

Note: The code attempts the requested matrix inverse and uses the pseudoinverse only if the matrix is singular.